WEEK 7 · GIT

Git & GitHub — 원격 저장소

GitHub에 원격 저장소 만들어 연결하기, 그리고 push · pull · clone으로 협업하기

목차

01 원격 저장소(Remote Repository)란?

02 GitHub에서 저장소 만들기

03 로컬 저장소 연결하기 (git remote)

04 인증(Authentication) — Personal Access Token(PAT)

05 push — 로컬 변경사항을 원격으로

06 clone — 원격 저장소 복제하기

07 태그(Tag)로 릴리스 지점 표시하기

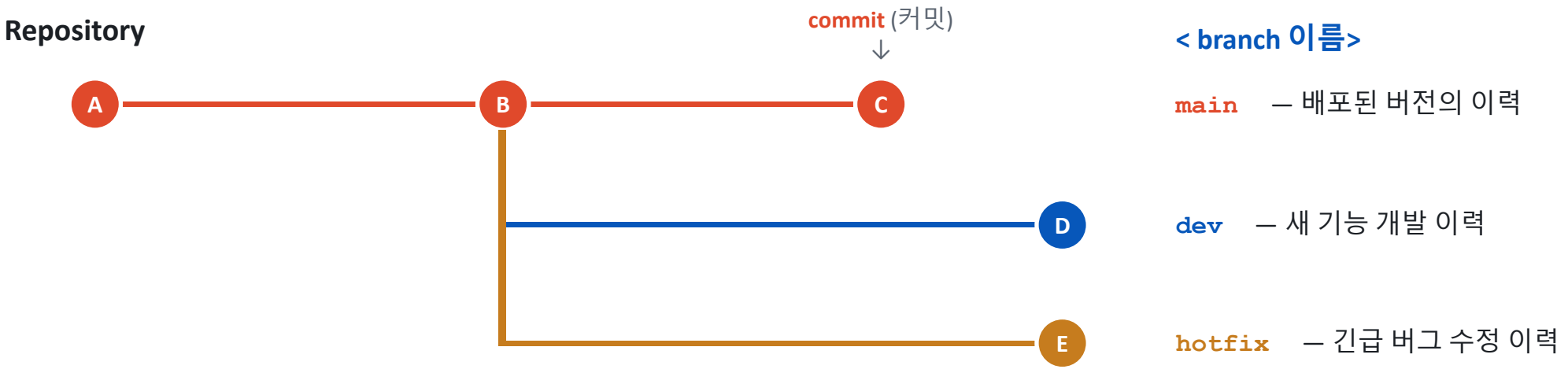
08 실습: 나의 저장소를 GitHub에 올리기

1.1

저장소 속 브랜치와 커밋, 다시 보기

하나의 저장소 안에서 브랜치와 커밋이 어떻게 구성되는지 복습합니다.

○ 표시(A~E)는 커밋(commit) 하나하나를, B 지점은 브랜치가 갈라져 나온(branch) 지점을 의미합니다.



핵심 커밋(commit)은 특정 시점의 스냅샷이고, 브랜치(branch)는 커밋이 이어진 독립적인 작업 흐름입니다.

1.2

로컬 저장소 vs 원격 저장소

지금까지는 내 컴퓨터(로컬) 안에서만 Git을 사용했습니다. 이제 GitHub 같은 원격 저장소와 연결합니다.

로컬 저장소 (Local)

내 컴퓨터(또는 실습용 EC2) 안의 .git 폴더

```
hello-git/
  .git/   (커밋 이력)
  Hello.java
  README.md
```

push →

← clone/pull

원격 저장소 (Remote)

GitHub 서버에 호스팅된 동일한 저장소

```
github.com/kim/hello-git
  (커밋 이력 전체 보관)
  Hello.java
  README.md
```

핵심 원격 저장소(remote repository)는 인터넷 어딘가에 호스팅된 Git 저장소입니다. GitHub · GitLab · Bitbucket이 대표적인 호스팅 서비스입니다.

2.1

GitHub 계정 만들기

github.com 에서 무료 계정을 만들고, 코드를 안전하게 지킬 보안 설정까지 함께 진행합니다.

1 **github.com 접속 → Sign up**

이메일 · 사용자명(username) · 비밀번호 입력 (이메일 인증 필요)

2 **사용자명(username) 확정**

이후 저장소 주소가 github.com/<username>/<repo> 형태로 생성됩니다

3 **2단계 인증(2FA) 설정**

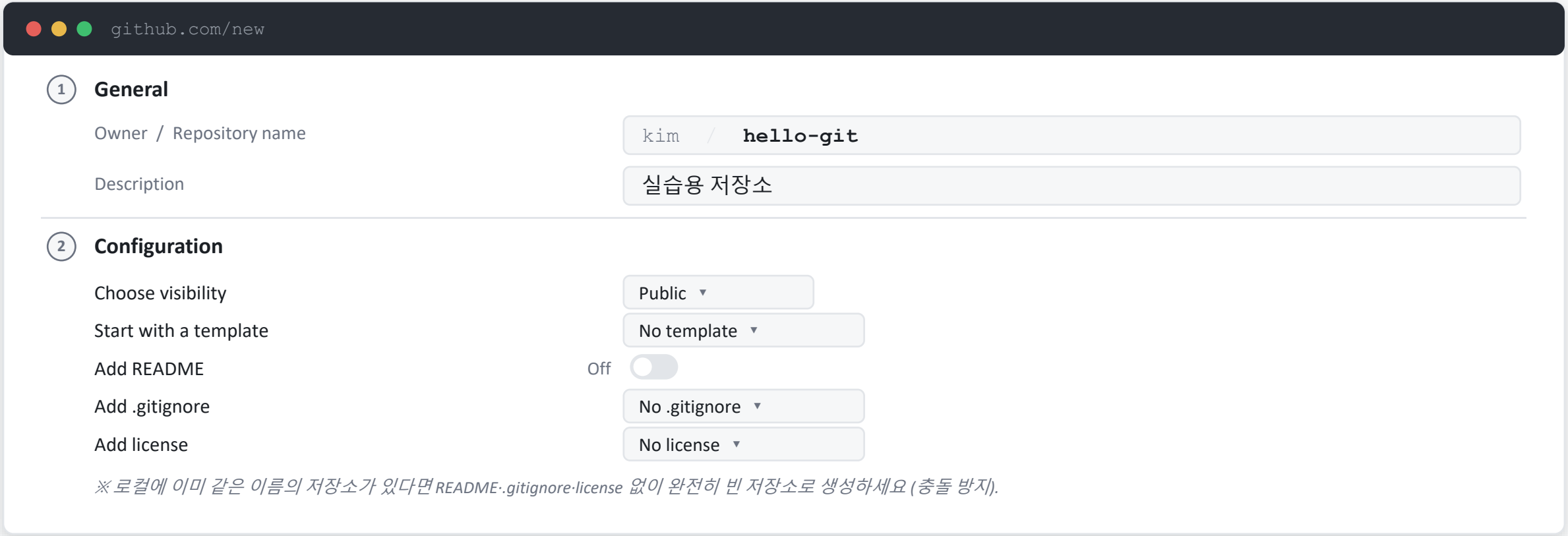
Settings → Password and authentication → Two-factor authentication

최신 팁 GitHub는 코드에 기여하거나 조직을 관리하는 계정에 2FA를 단계적으로 의무화하고 있습니다. 지금 미리 설정해두면 나중에 강제 전환으로 당황할 일이 없습니다.

2.2

새 원격 저장소 만들기 (New Repository)

GitHub 우측 상단의 + 버튼 → New repository 를 선택합니다.



핵심 2020년 10월부터 GitHub 저장소의 기본 브랜치명은 main 입니다.

3.1

git remote add — 원격 저장소에 이름 붙이기

로컬 저장소가 원격 저장소의 URL을 기억하도록 이름(별칭)을 등록합니다.

원격 저장소 연결하기 (HTTPS)

```
$ git remote add origin https://github.com/kim/hello-git.git
$ git remote -v
origin  https://github.com/kim/hello-git.git (fetch)
origin  https://github.com/kim/hello-git.git (push)
```

`git remote add <원격이름> <원격저장소 URL>`

핵심 origin은 관례적으로 쓰는 원격 저장소의 기본 별칭입니다. 반드시 이 이름이어야 하는 것은 아니지만, 대부분의 Git 문서와 도구가 origin을 기본값으로 가정합니다.

3.2

원격 연결 확인 · 이름 변경 · 해제

연결한 원격 저장소는 언제든 목록을 확인하고, 이름을 바꾸거나 제거할 수 있습니다.

연결 목록 확인

```
$ git remote -v
origin  https://github.com/kim/hello-git.git (fetch)
origin  https://github.com/kim/hello-git.git (push)
```

이름 변경 / 연결 해제

```
$ git remote rename origin upstream   # origin → upstream 으로 이름 변경
$ git remote remove upstream          # upstream 연결 해제
```

4.1

GitHub 인증 — 비밀번호 대신 PAT

2021년 8월부터 GitHub는 git push/pull 시 계정 비밀번호를 받지 않습니다. 대신 Personal Access Token(PAT)을 사용합니다.

GitHub 계정 비밀번호 (사용 불가)

- 2021년 8월부터 git push/pull 인증에 사용 불가
- 유출되면 계정 전체가 위험
- 권한 범위나 사용 기한을 제한할 수 없음
- 웹사이트 로그인에만 사용

Personal Access Token (PAT)

- 비밀번호 자리에 입력하는 긴 문자열 (ghp_로 시작)
- 필요한 권한(scope)만 골라서 발급
- 만료 기한을 정할 수 있고, 언제든 삭제 가능
- 유출돼도 토큰만 삭제하면 계정은 안전

4.2

Personal Access Token(PAT) 발급하기

Fine-grained 토큰은 Collaborator로 초대받은 저장소에 push할 수 없으므로, 이번 실습은 Alice · Bob 모두 classic 토큰을 사용합니다.

1. Settings → Developer settings → Personal access tokens
2. Tokens (classic) → Generate new token → Generate new token (classic)
3. 아래 화면에서 Note · Expiration · Select scopes를 설정합니다

Note · Expiration

- Note: 토큰 용도 메모 (예: git-class)
- Expiration: 7 days 등 짧게 설정
- Custom을 고르면 실습 종료일로 지정 가능

Select scopes

- repo 항목 아래 public_repo 만 체크
  → 공개 저장소에 push 가능 · Private 저장소는 접근 불가
- Private 저장소라면 repo 전체 체크
  → 내가 접근 가능한 모든 저장소(Private 포함)에 전체 권한

4. Generate token 클릭 → 표시된 토큰 값을 즉시 복사 (다시 보여주지 않음)

주의 PAT는 비밀번호와 같습니다. 코드에 커밋하거나 다른 사람과 공유하지 마세요 — 공유받은 사람은 토큰 주인으로 접속하게 됩니다.

4.3

PAT로 push/pull 인증하기

push나 pull 시 비밀번호를 묻는 화면에 발급받은 PAT를 붙여넣습니다.

PAT로 인증

```
$ git push origin main
Username: kim
Password: <PAT를 붙여넣기, 화면에 표시되지 않음>
```

PAT를 저장해 다음부터 자동으로 인증되도록 설정 (본인만 쓰는 PC·서버에서만)

```
$ git config --global credential.helper store
```

최신 팁 credential.helper store는 토큰을 평문 파일에 저장합니다. 암호화된 OS 보안 저장소를 쓰려면 Windows는 Git Credential Manager(GCM), macOS는 osxkeychain, Linux는 libsecret을 선택하세요.

5.1

git push — 브랜치를 원격 저장소로

git push <원격이름> <로컬브랜치이름> — 로컬 브랜치의 커밋을 원격 저장소에 반영합니다.

push

```
$ git push origin main
# 로컬 main 브랜치 → 원격 origin/main 브랜치
```



5.2

-u 옵션 — upstream 브랜치 설정

-u(--set-upstream)는 로컬 브랜치와 원격 브랜치를 연결해, 이후 명령을 짧게 줄여줍니다.

최초 push (upstream 설정)

```
$ git push -u origin main
Branch 'main' set up to track 'origin/main'.
```

이후에는 이름 없이 사용 가능

```
$ git push
$ git pull
```

최신 팁 -u 옵션은 처음 한 번만 붙이면 됩니다. 이후에는 git push, git pull만으로 origin의 main 브랜치와 동기화됩니다.

6.1

git clone — 저장소 통째로 복제하기

원격 저장소의 커밋 이력 전체를 새 디렉터리로 내려받습니다.

HTTPS로 clone (새 디렉터리에)

```
$ git clone https://github.com/kim/hello-git.git hello-git-cloned
```

clone된 저장소의 원격 연결 확인

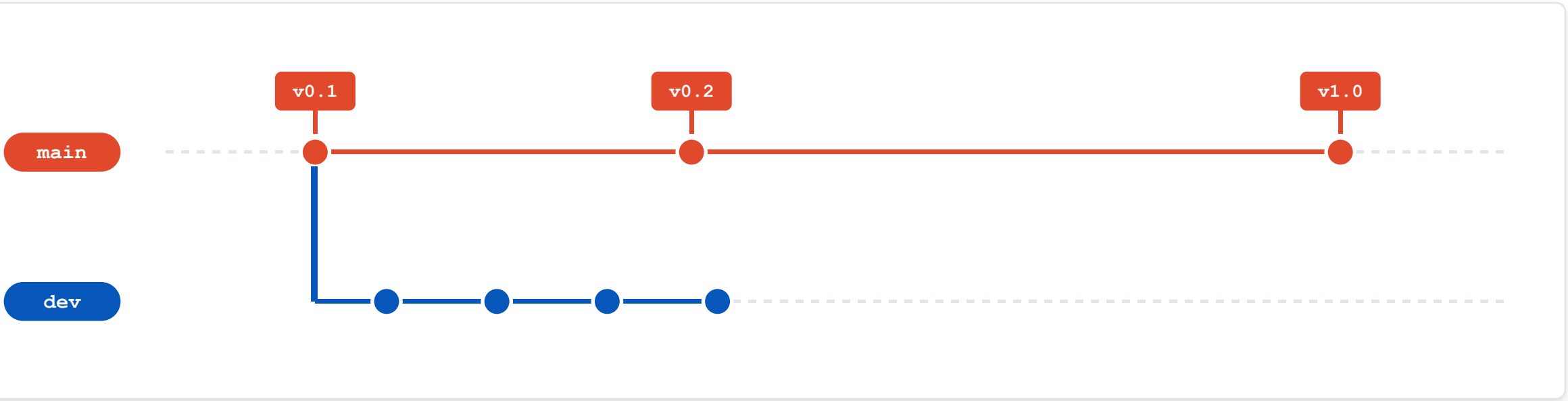
```
$ cd hello-git-cloned
$ git remote -v
origin  https://github.com/kim/hello-git.git (fetch)
```

핵심 git clone으로 만든 디렉터리는 이미 origin이라는 이름으로 원격이 자동 연결되어 있습니다. 로컬의 기존 hello-git과 겹치지 않도록 디렉터리 이름을 hello-git-cloned로 지정했습니다.

7.1

태그(Tag)로 릴리스 지점 표시하기

태그는 특정 커밋에 붙이는 '이름표'로, 주로 릴리스 버전(v1.0.0 등)을 표시할 때 사용합니다.



핵심 태그는 브랜치와 달리 위치가 고정됩니다. 코드를 자주 커밋·push해 백업하고, 기능이 안정된 지점에서 태그를 남기는 것이 좋은 습관입니다.

7.2

Annotated Tag 만들기 (git tag -a)

-a 옵션으로 만든 태그는 작성자 · 날짜 · 메시지가 함께 저장되는 '주석 태그'입니다.

최신 커밋에 태그 달기

```
$ git tag -a v1.0.0 -m "First stable release"
```

특정(과거) 커밋에 태그 달기

```
$ git tag -a v0.9.0 a1b2c3d -m "Beta release"
# a1b2c3d 자리에 커밋 해시를 지정
```

핵심 Git 데이터베이스에는 태그를 만든 사람, 날짜, 태그 메시지, 그리고 태그가 가리키는 커밋 정보가 함께 저장됩니다.

7.3

태그 목록 조회하기

만들어 둔 태그의 목록과 상세 정보를 확인합니다.

태그 목록 보기

```
$ git tag
v0.9.0
v1.0.0
```

태그 상세 정보 보기

```
$ git show v1.0.0
tag v1.0.0
Tagger: Kim <kim@example.com>
Date:   Wed Sep 30 14:00:00 2026 +0900
First stable release
```

7.4

태그를 원격 저장소에 push하기

git push만으로는 태그가 원격에 올라가지 않습니다 — 태그는 별도로 push해야 합니다.

태그 하나만 push

```
$ git push origin v1.0.0
```

모든 태그를 한 번에 push

```
$ git push origin --tags
```

핵심 GitHub 저장소의 Releases 메뉴에서, push된 태그를 바탕으로 릴리스 노트를 작성할 수 있습니다.

정리하기

1. 원격 저장소는 GitHub 같은 서버에 호스팅된 Git 저장소이며, git remote add로 로컬에 이름(origin)을 붙여 연결합니다.
2. 2021년 8월부터 GitHub는 비밀번호 인증을 지원하지 않습니다 — 비밀번호 자리에 Personal Access Token(PAT)을 입력합니다.
3. git push <원격> <브랜치>로 올리고, -u 옵션으로 upstream을 설정하면 이후 git push / git pull만으로 충분합니다.
4. git clone은 원격 저장소 전체를 복제하며, 복제된 저장소에는 origin이 자동으로 연결됩니다.
5. git tag -a로 릴리스 지점을 표시하고, git push origin --tags로 원격에 반영합니다.
6. 다음 시간에는 브랜치(Branch)와 병합(Merge), 충돌 해결을 다룹니다.

08

실습: 나의 저장소를 GitHub에 올리기

Alice → Bob 순서로 진행합니다 (2인 1조, 역할은 서로 바꿔서 다시 해봐도 좋습니다).

역할 / 단계	실습 내용
Alice: 계정 생성	GitHub 회원가입 및 2FA 설정
Alice: 원격 저장소 생성	New repository로 hello-git 생성 — README/.gitignore 없이, 기본 브랜치 main
Alice: 인증 준비	classic PAT 발급 (public_repo 권한, 짧은 만료)
Alice: 로컬 연결 & push	git branch -M main → git remote add origin ... → git push -u origin main
Alice: Bob 초대	Settings → Collaborators → Add people 로 Bob 초대 (Bob은 메일·알림에서 수락)
Bob: 저장소 clone	git clone https://github.com/kim/hello-git.git (Public 저장소라 인증 불필요)
Bob: 변경 후 push	Bob도 classic PAT 발급 → 파일 수정 → git add → git commit → git push (PAT로 인증)
Alice: 최신 변경 받기	git pull 로 Bob의 변경사항 확인